IMPORTING LIBRARIES

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

DATA PROCESSING AND EDA

In [3]:
```
df = pd.read_csv(r'UCI_Credit_Card.csv')
```

In [4]:
```
df.head()
```

Out [4]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0.0	0.0	689.0	0.0
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261.0	0.0	1000.0	1000.0
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549.0	1518.0	1500.0	1000.0
3	4	50000.0	2	2	1	37	0	0	0	0	...	28314.0	28959.0	29547.0	2000.0	2019.0	1200.0
4	5	50000.0	1	2	1	57	-1	0	-1	0	...	20940.0	19146.0	19131.0	2000.0	36681.0	10000.0

5 rows × 25 columns

In [42]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 34 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   LIMIT_BAL                   30000 non-null  float64
 1   SEX                         30000 non-null  int64
 2   EDUCATION                   30000 non-null  int64
 3   MARRIAGE                    30000 non-null  int64
 4   AGE                         30000 non-null  int64
 5   PAY_0                       30000 non-null  int64
 6   PAY_2                       30000 non-null  int64
 7   PAY_3                       30000 non-null  int64
 8   PAY_4                       30000 non-null  int64
 9   PAY_5                       30000 non-null  int64
 10  PAY_6                       30000 non-null  int64
 11  BILL_AMT1                   30000 non-null  float64
 12  BILL_AMT2                   30000 non-null  float64
 13  BILL_AMT3                   30000 non-null  float64
 14  BILL_AMT4                   30000 non-null  float64
 15  BILL_AMT5                   30000 non-null  float64
 16  BILL_AMT6                   30000 non-null  float64
 17  PAY_AMT1                    30000 non-null  float64
 18  PAY_AMT2                    30000 non-null  float64
 19  PAY_AMT3                    30000 non-null  float64
 20  PAY_AMT4                    30000 non-null  float64
 21  PAY_AMT5                    30000 non-null  float64
 22  PAY_AMT6                    30000 non-null  float64
 23  default.payment.next.month  30000 non-null  int64
 24  credit_utilization          30000 non-null  float64
 25  avg_pay_delay               30000 non-null  float64
 26  max_delay                   30000 non-null  int64
 27  total_bill                  30000 non-null  float64
 28  total_payment               30000 non-null  float64
 29  payment_efficiency          29205 non-null  float64
 30  debt_growth                 30000 non-null  float64
 31  late_payment_count          30000 non-null  int64
 32  high_risk_flag              30000 non-null  int64
 33  financial_stress_score      30000 non-null  float64
dtypes: float64(20), int64(14)
memory usage: 7.8 MB
```

In [43]:
```
df.describe()
```

Out [43]:

	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	...	cr
count	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	30000.000000	...	30
mean	167484.322667	1.603733	1.842267	1.557267	35.485500	-0.016700	-0.133767	-0.166200	-0.220667	-0.266200	...	0.
std	129747.661567	0.489129	0.744494	0.521405	9.217904	1.123802	1.197186	1.196868	1.169139	1.133187	...	0.
min	10000.000000	1.000000	1.000000	1.000000	21.000000	-2.000000	-2.000000	-2.000000	-2.000000	-2.000000	...	-0
25%	50000.000000	1.000000	1.000000	1.000000	28.000000	-1.000000	-1.000000	-1.000000	-1.000000	-1.000000	...	0.
50%	140000.000000	2.000000	2.000000	2.000000	34.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.
75%	240000.000000	2.000000	2.000000	2.000000	41.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.
max	1000000.000000	2.000000	4.000000	3.000000	79.000000	8.000000	8.000000	8.000000	8.000000	8.000000	...	6.

8 rows × 34 columns

In [44]:
```
df.isnull().sum()
```

Out [44]:
```
LIMIT_BAL                     0
SEX                           0
EDUCATION                     0
MARRIAGE                      0
AGE                           0
PAY_0                         0
PAY_2                         0
PAY_3                         0
PAY_4                         0
PAY_5                         0
PAY_6                         0
BILL_AMT1                     0
```

```
BILL_AMT2                       0
BILL_AMT3                       0
BILL_AMT4                       0
BILL_AMT5                       0
BILL_AMT6                       0
PAY_AMT1                        0
PAY_AMT2                        0
PAY_AMT3                        0
PAY_AMT4                        0
PAY_AMT5                        0
PAY_AMT6                        0
default.payment.next.month      0
credit_utilization              0
avg_pay_delay                   0
max_delay                       0
total_bill                      0
total_payment                   0
payment_efficiency            795
debt_growth                     0
late_payment_count              0
high_risk_flag                  0
financial_stress_score          0
dtype: int64
```

In [45]:
```
print(df.duplicated().sum())
```

```
35
```

In [51]:
```
df.nunique()
```

Out [51]:
```
LIMIT_BAL                      81
SEX                             2
EDUCATION                       4
MARRIAGE                        3
AGE                            56
PAY_0                          11
PAY_2                          11
PAY_3                          11
PAY_4                          11
PAY_5                          10
PAY_6                          10
BILL_AMT1                   22723
BILL_AMT2                   22346
BILL_AMT3                   22026
BILL_AMT4                   21548
BILL_AMT5                   21010
BILL_AMT6                   20604
PAY_AMT1                     7943
PAY_AMT2                     7899
PAY_AMT3                     7518
PAY_AMT4                     6937
PAY_AMT5                     6897
PAY_AMT6                     6939
default.payment.next.month      2
credit_utilization          25565
avg_pay_delay                  47
max_delay                      11
total_bill                  27370
total_payment               19180
payment_efficiency          27650
debt_growth                 21836
late_payment_count              7
high_risk_flag                  2
financial_stress_score      19188
dtype: int64
```

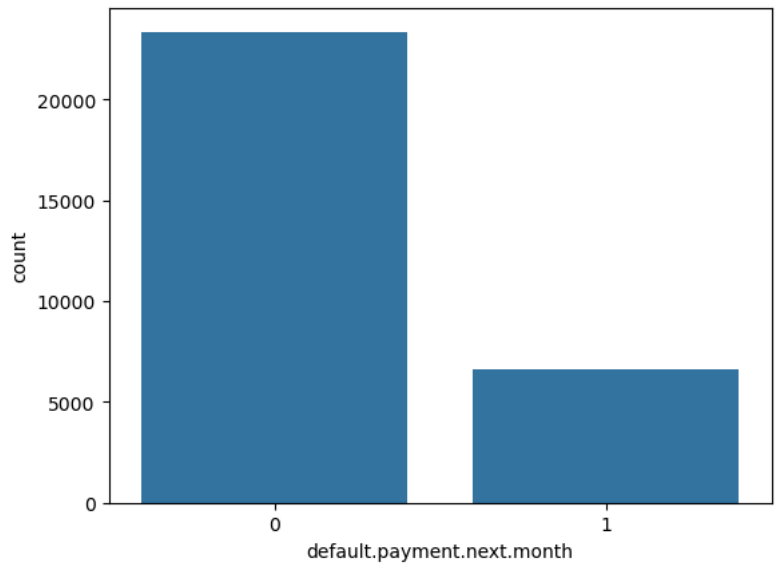
In [10]:
```
df.shape
```

Out [10]:
```
(30000, 25)
```

In [47]:
```
df['default.payment.next.month'].value_counts()
```

Out [47]:
```
default.payment.next.month
0    23364
1     6636
Name: count, dtype: int64
```

In [48]:
```
sns.countplot(x='default.payment.next.month', data=df)
plt.show()
```



In [54]:
```
print(df['SEX'].unique())
print(df['EDUCATION'].unique())
print(df['MARRIAGE'].unique())
```

```
[2 1]
[2 1 3 4]
[1 2 3]
```

In [53]:
```
df['EDUCATION'] = df['EDUCATION'].replace([0,5,6], 4)
df['MARRIAGE'] = df['MARRIAGE'].replace(0, 3)
```

In [52]:

```
# 1) CREDIT UTILIZATION RATIO

df["credit_utilization"] = df["BILL_AMT1"] / df["LIMIT_BAL"]
df["credit_utilization"] = df["credit_utilization"].replace([np.inf, -np.inf], 0)


# 2) AVERAGE PAYMENT DELAY

df["avg_pay_delay"] = (
    df["PAY_0"] +
    df["PAY_2"] +
    df["PAY_3"] +
    df["PAY_4"] +
    df["PAY_5"] +
    df["PAY_6"]
) / 6


# 3) MAXIMUM DELAY

df["max_delay"] = df[
    ["PAY_0", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6"]
].max(axis=1)


# 4) TOTAL BILL AMOUNT


df["total_bill"] = (
    df["BILL_AMT1"] +
    df["BILL_AMT2"] +
    df["BILL_AMT3"] +
    df["BILL_AMT4"] +
    df["BILL_AMT5"] +
    df["BILL_AMT6"]
)


# 5) TOTAL PAYMENT AMOUNT

df["total_payment"] = (
    df["PAY_AMT1"] +
    df["PAY_AMT2"] +
    df["PAY_AMT3"] +
    df["PAY_AMT4"] +
    df["PAY_AMT5"] +
    df["PAY_AMT6"]
)


# 6) PAYMENT EFFICIENCY

df["payment_efficiency"] = (
    df["total_payment"] / df["total_bill"]
)

df["payment_efficiency"] = df["payment_efficiency"].replace([np.inf, -np.inf], 0)


# 7) DEBT GROWTH

df["debt_growth"] = df["BILL_AMT1"] - df["BILL_AMT6"]

# 8) LATE PAYMENT COUNT


pay_cols = ["PAY_0", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6"]

df["late_payment_count"] = (df[pay_cols] > 0).sum(axis=1)


# 9) HIGH RISK FLAG

df["high_risk_flag"] = np.where(
    (df["avg_pay_delay"] > 2) &
    (df["credit_utilization"] > 0.8),
    1,
    0
)


# 10) FINANCIAL STRESS SCORE

df["financial_stress_score"] = (
    (df["avg_pay_delay"] * 15) +
    (df["credit_utilization"] * 40) +
    (df["late_payment_count"] * 5)
)

# Normalize score
df["financial_stress_score"] = df["financial_stress_score"].clip(0, 100)
```

```
print("Feature Engineering Completed Successfully ")
```

```
Feature Engineering Completed Successfully
```

In [31]:
```
# CHECK NEW FEATURES


new_features = [
    "credit_utilization",
    "avg_pay_delay",
    "max_delay",
    "total_bill",
    "total_payment",
    "payment_efficiency",
    "debt_growth",
    "late_payment_count",
    "high_risk_flag",
    "financial_stress_score"
]

print(df[new_features].head())
```

```
   credit_utilization  avg_pay_delay  max_delay  total_bill  total_payment  \
0            0.195650      -0.333333          2      7704.0          689.0
1            0.022350       0.500000          2     17077.0         5000.0
2            0.324878       0.000000          0    101653.0        11018.0
3            0.939800       0.000000          0    231334.0         8388.0
4            0.172340      -0.333333          0    109339.0        59049.0

   payment_efficiency  debt_growth  late_payment_count  high_risk_flag  \
0            0.089434       3913.0                   2               0
1            0.292791       -579.0                   2               0
2            0.108388      13690.0                   0               0
3            0.036259      17443.0                   0               0
4            0.540054     -10514.0                   0               0

   financial_stress_score
0               12.826000
1               18.394000
2               12.995111
3               37.592000
4                1.893600
```

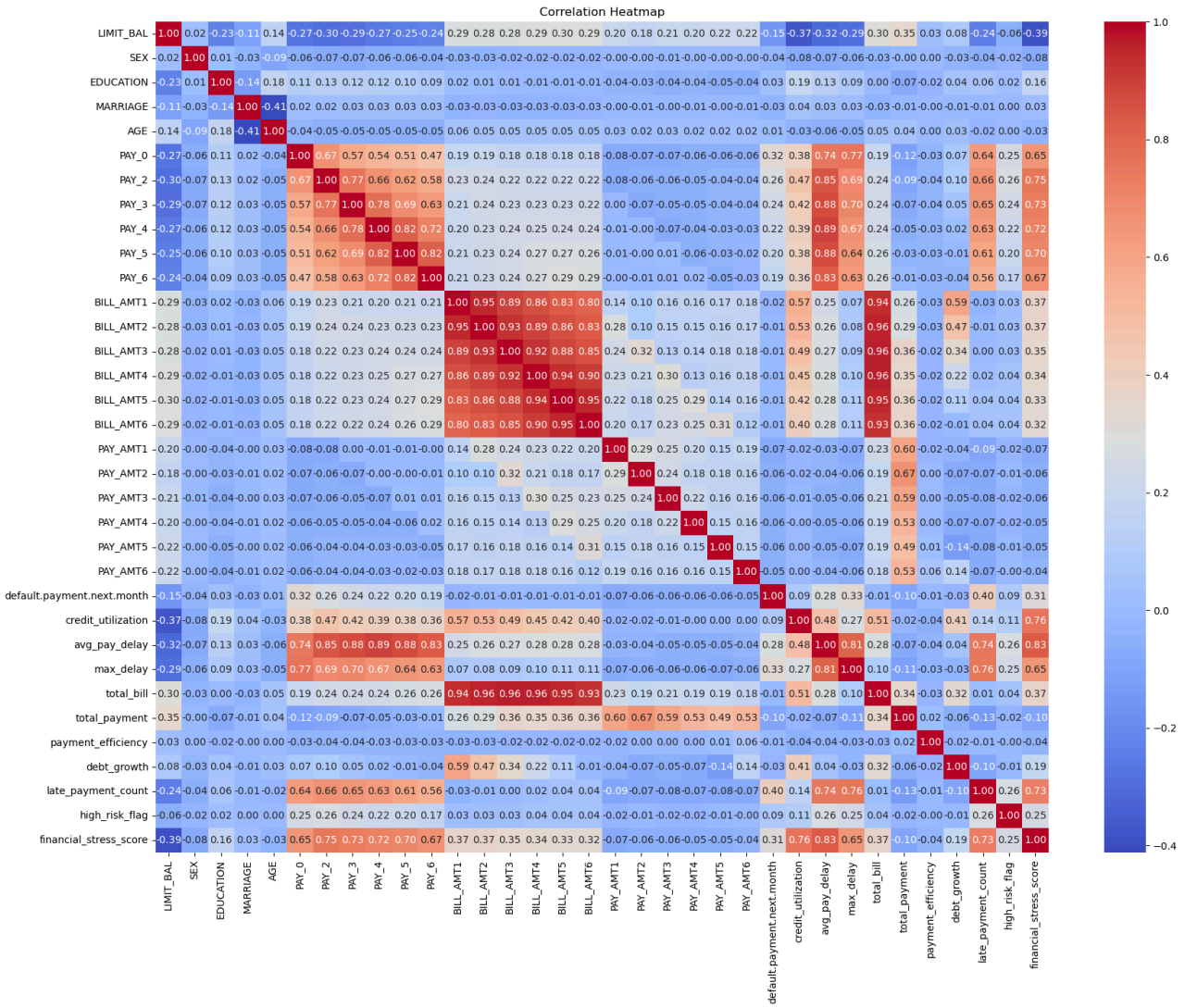
In [56]:
```
df.isnull().sum()
```

Out [56]:
```
LIMIT_BAL                       0
SEX                             0
EDUCATION                       0
MARRIAGE                        0
AGE                             0
PAY_0                           0
PAY_2                           0
PAY_3                           0
PAY_4                           0
PAY_5                           0
PAY_6                           0
BILL_AMT1                       0
BILL_AMT2                       0
BILL_AMT3                       0
BILL_AMT4                       0
BILL_AMT5                       0
BILL_AMT6                       0
PAY_AMT1                        0
PAY_AMT2                        0
PAY_AMT3                        0
PAY_AMT4                        0
PAY_AMT5                        0
PAY_AMT6                        0
default.payment.next.month      0
credit_utilization              0
avg_pay_delay                   0
max_delay                       0
total_bill                      0
total_payment                   0
payment_efficiency            795
debt_growth                     0
late_payment_count              0
high_risk_flag                  0
financial_stress_score          0
dtype: int64
```

In [57]:
```
df.dropna(inplace=True)
```

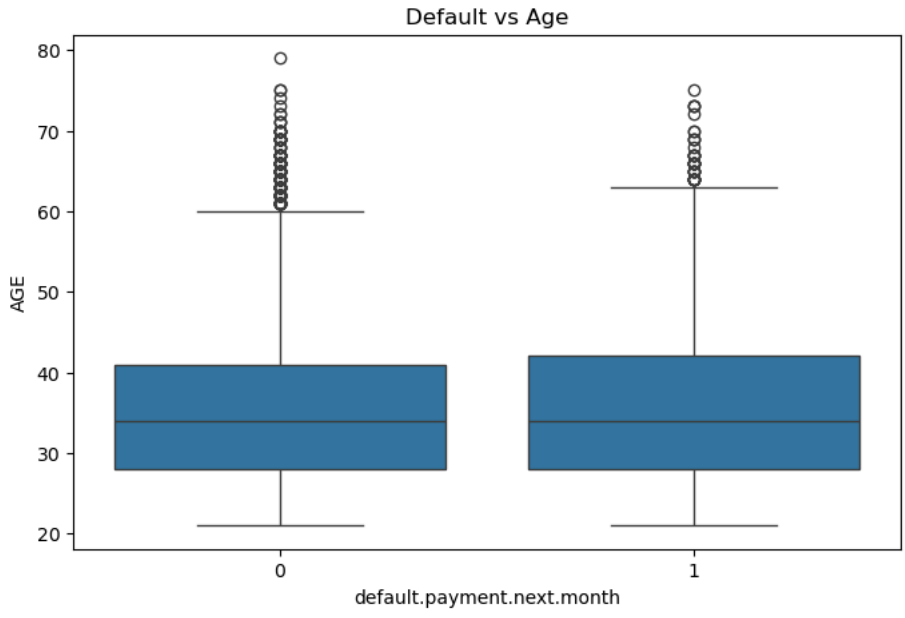
DATA VISUALIZATION

In [33]:
```
plt.figure(figsize=(20,15))
sns.heatmap(df.corr(),annot=True,fmt='.2f', cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



In [34]:

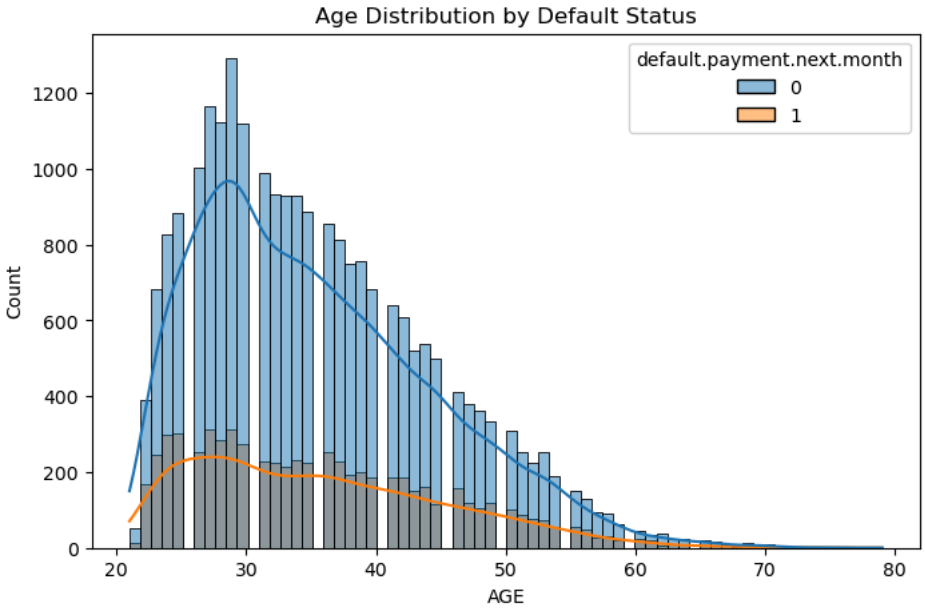
```
plt.figure(figsize=(8,5))

sns.boxplot(
    x='default.payment.next.month',
    y='AGE',
    data=df
)

plt.title("Default vs Age")
plt.show()
```



In [35]:

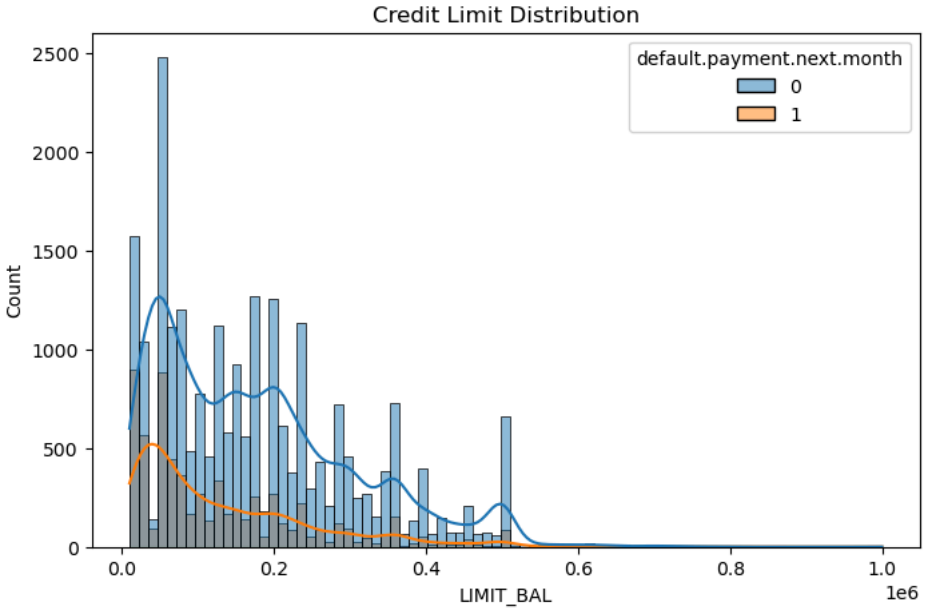
```
plt.figure(figsize=(8,5))

sns.histplot(
    data=df,
    x='AGE',
    hue='default.payment.next.month',
    kde=True
```

```
)

plt.title("Age Distribution by Default Status")
plt.show()
```



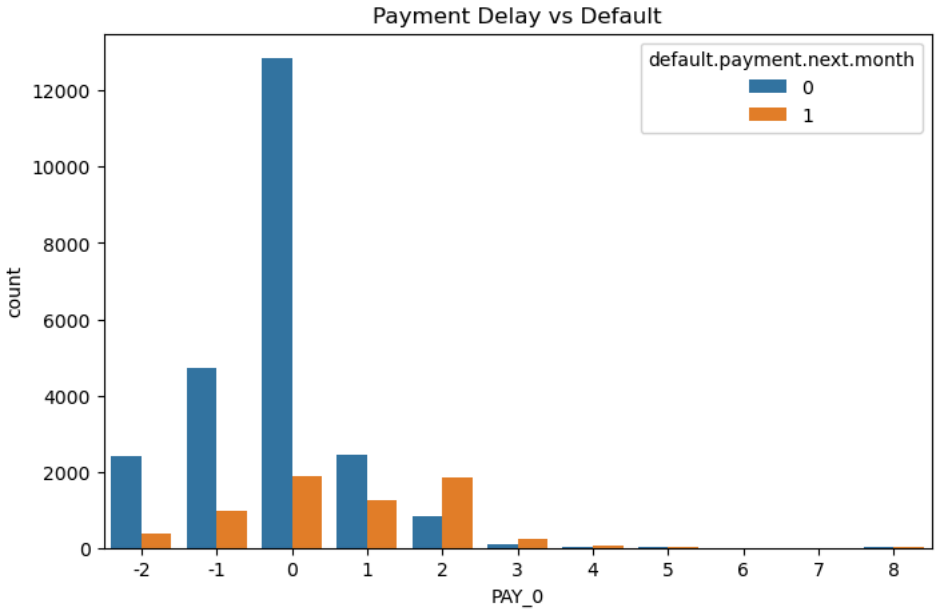
In [19]:

```
plt.figure(figsize=(8,5))

sns.histplot(
    data=df,
    x='LIMIT_BAL',
    hue='default.payment.next.month',
    kde=True
)

plt.title("Credit Limit Distribution")
plt.show()
```



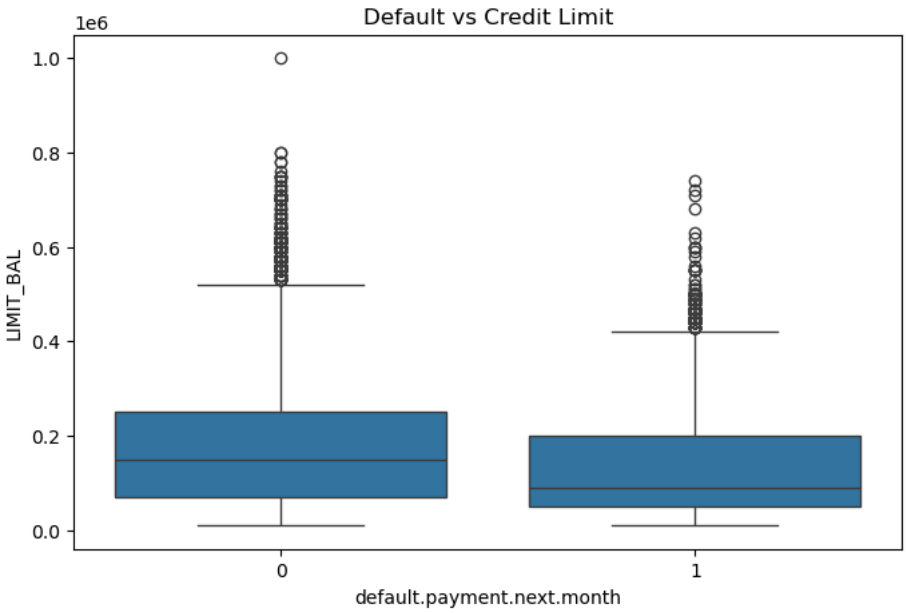
In [36]:

```
plt.figure(figsize=(8,5))

sns.countplot(
    x='PAY_0',
    hue='default.payment.next.month',
    data=df
)

plt.title("Payment Delay vs Default")
plt.show()
```



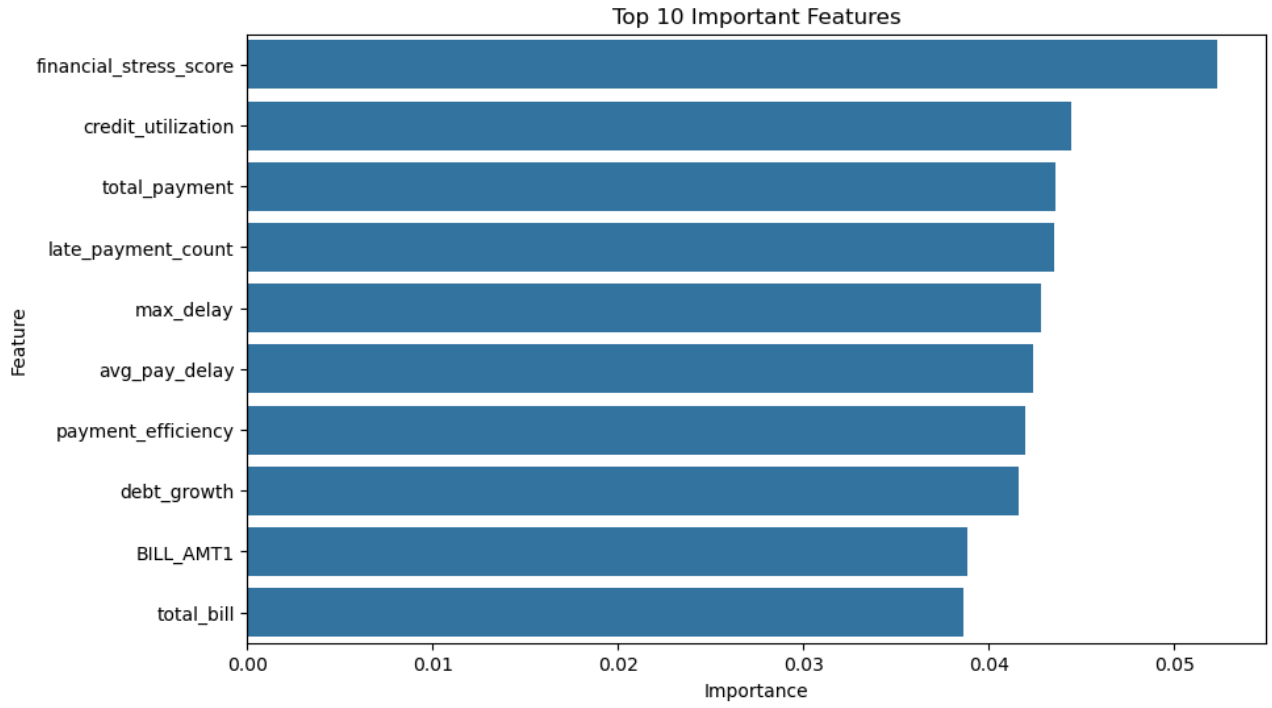
In [37]:

```
plt.figure(figsize=(8,5))

sns.boxplot(
    x='default.payment.next.month',
    y='LIMIT_BAL',
    data=df
)

plt.title("Default vs Credit Limit")
plt.show()
```



In [86]:

```
top_features = feature_importance.head(10)

plt.figure(figsize=(10,6))

sns.barplot(
    x='Importance',
    y='Feature',
   data=top_features
)

plt.title("Top 10 Important Features")
plt.show()
```



SPLITTING

In [59]:
```
X = df.drop('default.payment.next.month', axis=1)

y = df['default.payment.next.month']
```

In [60]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

In [61]:
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

LOGISTIC REGRESSION

In [62]:
```
log_model = LogisticRegression(class_weight='balanced')

log_model.fit(X_train_scaled, y_train)
```

Out [62]:
```
LogisticRegression
LogisticRegression(class_weight='balanced')
```

In [63]:
```
y_pred_log = log_model.predict(X_test_scaled)
```

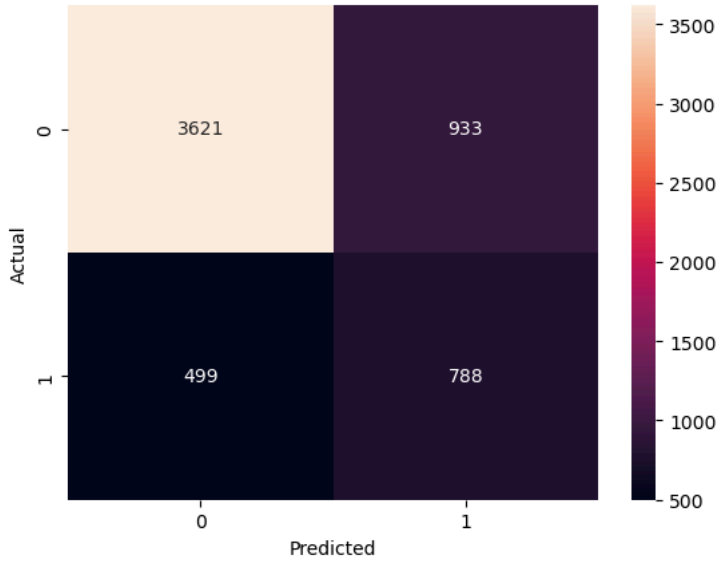
In [65]:
```
log_accuracy = accuracy_score(y_test, y_pred_log)
print (log_accuracy)
```

```
0.7548365005992125
```

In [66]:
```
#Recall is especially important in this project because failing to identify a risky customer could lead to financial loss
print(classification_report(y_test, y_pred_log))
```

```
              precision    recall  f1-score   support

           0       0.88      0.80      0.83      4554
           1       0.46      0.61      0.52      1287

    accuracy                           0.75      5841
   macro avg       0.67      0.70      0.68      5841
weighted avg       0.79      0.75      0.77      5841
```

In [67]:
```
cm = confusion_matrix(y_test, y_pred_log)

sns.heatmap(cm,annot=True,fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



RANDOM FOREST

In [68]:
```
rf_model = RandomForestClassifier(
    n_estimators=100,
    class_weight='balanced',
    random_state=42
)

rf_model.fit(X_train, y_train)
```

Out [68]:
```
RandomForestClassifier(class_weight='balanced', random_state=42)
```

In [69]:
```
y_pred_rf = rf_model.predict(X_test)

print(classification_report(y_test, y_pred_rf))
```

```
              precision    recall  f1-score   support

           0       0.84      0.95      0.89      4554
           1       0.68      0.35      0.46      1287

    accuracy                           0.82      5841
   macro avg       0.76      0.65      0.68      5841
weighted avg       0.80      0.82      0.80      5841
```

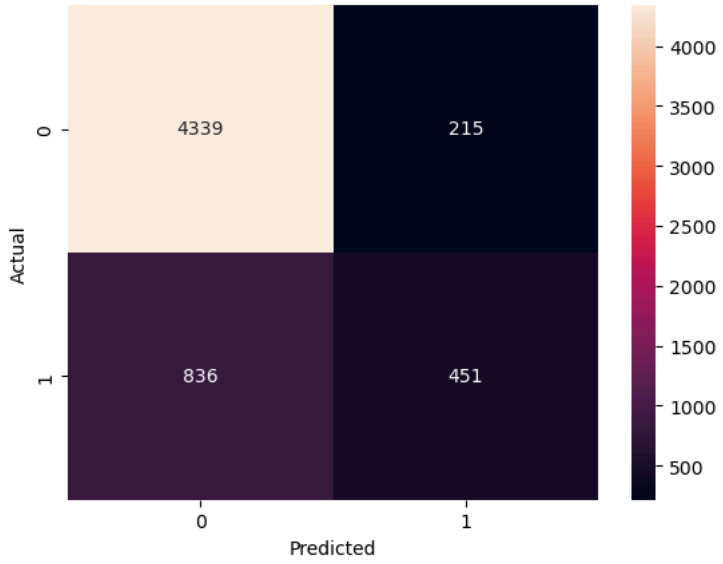
In [70]:
```
rf_accuracy= accuracy_score(y_test, y_pred_rf)
print(rf_accuracy)
```

```
0.8200650573531929
```

In [71]:
```
importance = rf_model.feature_importances_

feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance
})

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

print(feature_importance.head(10))
```

```
                  Feature  Importance
32  financial_stress_score    0.052352
23     credit_utilization    0.044472
27          total_payment    0.043621
30     late_payment_count    0.043520
25              max_delay    0.042819
24          avg_pay_delay    0.042414
28     payment_efficiency    0.041967
29           debt_growth    0.041590
11              BILL_AMT1    0.038871
26             total_bill    0.038654
```

In [72]:
```
cm = confusion_matrix(y_test, y_pred_rf)

sns.heatmap(cm,annot=True,fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



SVM

In [73]:
```
svm_model = SVC(class_weight='balanced')

svm_model.fit(X_train_scaled, y_train)
```

Out [73]:
```
SVC(class_weight='balanced')
```

In [74]:
```
y_pred_svm = svm_model.predict(X_test_scaled)
```

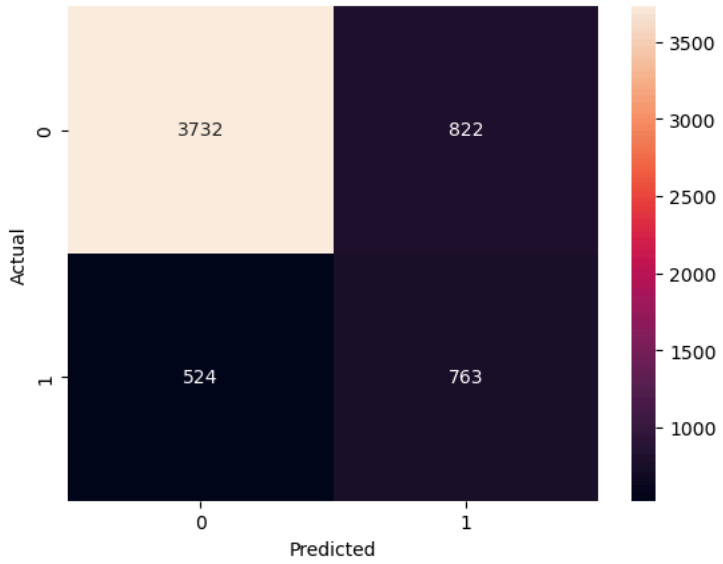
In [76]:
```
print(classification_report(y_test, y_pred_svm))
```

```
              precision    recall  f1-score   support

           0       0.88      0.82      0.85      4554
           1       0.48      0.59      0.53      1287

    accuracy                           0.77      5841
   macro avg       0.68      0.71      0.69      5841
weighted avg       0.79      0.77      0.78      5841
```

In [77]:
```
svm_accuracy = accuracy_score(y_test, y_pred_svm)

print(svm_accuracy)
```

```
0.7695600068481424
```

In [78]:
```
cm = confusion_matrix(y_test, y_pred_svm)

sns.heatmap(cm,annot=True,fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



FINAL COMPARISON

In [79]:
```
comparison = pd.DataFrame({
    'Model': [
        'Logistic Regression',
```

```
        'Random Forest',
        'SVM'
    ],

    'Accuracy': [
        log_accuracy,
        rf_accuracy,
        svm_accuracy
    ]
})

print(comparison)
```

```
                Model  Accuracy
0  Logistic Regression  0.754837
1        Random Forest  0.820065
2                  SVM  0.769560
```

In [101]:

```
comparison.plot(
    x='Model',
    y='Accuracy',
    kind='bar',
    legend=False
)

plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```